



UNIVERSIDAD  
**NACIONAL**  
DE COLOMBIA

---

Universidad Nacional de Colombia - Sede Medellín

## **Actividad 2: Programación Orientada a Objetos**

Estudiante: Victor Manuel Rivas Morales

Docente: Walter Hugo Arboleda Mazo

Medellín, 2026

## Ejercicio 2.1

### Codigo:

```
class Persona:
    def __init__(self, nombre, apellidos, id, yob):
        self.nombre = nombre
        self.apellidos = apellidos
        self.id = id
        self.yob = yob

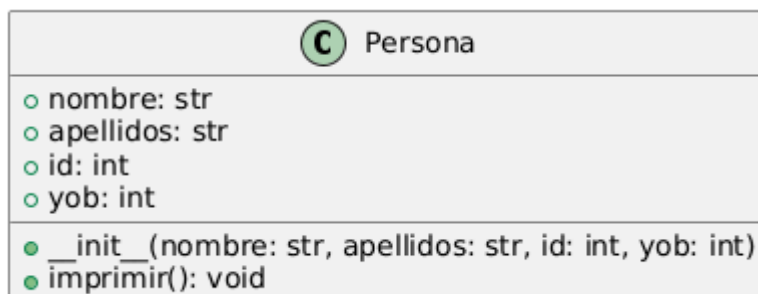
    def imprimir(self):
        print(f'Nombre = {self.nombre}\nApellidos = {self.apellidos}\nNumero de documento de identidad = {self.id}\nAno de nacimiento = {self.yob}')

def main():
    p1 = Persona('Rochi', 'Roldan Laverde', 1010104101, 2005)
    p1.imprimir()

    p2 = Persona('Zype', 'Monsalve Mazo', 1089893799, 2003)
    p2.imprimir()

if __name__ == "__main__":
    main()
```

### Diagrama de clase:



## Ejercicio 2.2

### Codigo:

```
from enum import Enum
```

```

class TipoPlaneta(Enum):
    GASEOSO = "GASEOSO"
    TERRESTRE = "TERRESTRE"
    ENANO = "ENANO"

class Planeta:
    def __init__(self, nombre=None, satelites=0, masa=0, volumen=0,
diametro=0, dsol=0, tipo=None, observable=False):
        self.nombre = nombre
        self.satelites = satelites
        self.masa = masa
        self.volumen = volumen
        self.diametro = diametro
        self.dsol = dsol
        self.tipo = tipo
        self.observable = observable

    def imprimir(self):
        print(f"Nombre del planeta = {self.nombre}")
        print(f"Cantidad de satélites = {self.satelites}")
        print(f"Masa del planeta = {self.masa}")
        print(f"Volumen del planeta = {self.volumen}")
        print(f"Diámetro del planeta = {self.diametro}")
        print(f"Distancia al sol = {self.dsol}")
        print(f"Tipo de planeta = {self.tipo.name}")
        print(f"Es observable = {self.observable}")

    def densidad(self):
        return self.masa / self.volumen if self.volumen != 0 else 0

    def planetaExterior(self):
        limite = 508632758
        return self.dsol > limite

def main():
    marte = Planeta('Tierra', 1, 5.9736E24, 1.08321E12, 12742,
150000000, TipoPlaneta.TERRESTRE, True)
    marte.imprimir()
    print(f'Es exterior = {marte.planetaExterior()}')

```

```

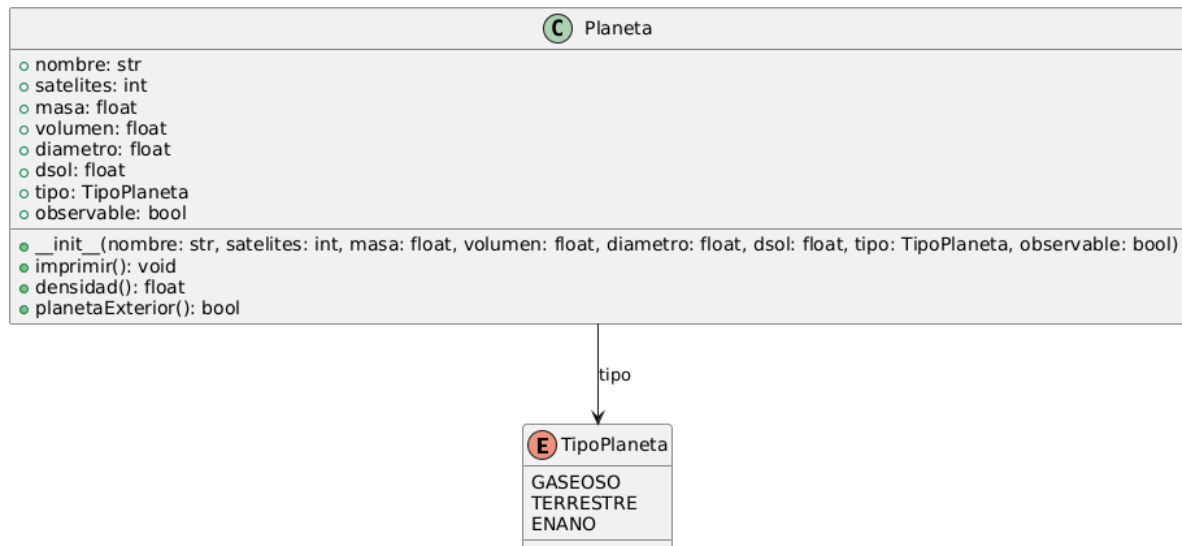
print(f'Densidad = {marte.densidad()}')

jupiter = Planeta('Jupiter', 79, 1.899E27, 1.4313E15, 139820,
7500000000, TipoPlaneta.GASEOSO, True)
jupiter.imprimir()
print(f'Es exterior = {jupiter.planetaExterior()}')
print(f'Densidad = {jupiter.densidad()}')

if __name__ == "__main__":
    main()

```

### Diagrama de clase:



### Ejercicio 2.3

#### Codigo:

```

from enum import Enum

class TipoCom(Enum):
    GASOLINA = "GASOLINA"
    BIOETANOL = "BIOETANOL"
    DIESEL = "DIESEL"
    BIODIESEL = "BIODIESEL"
    GAS_NATURAL = "GAS_NATURAL"

```

```
class TipoA(Enum):
    CIUDAD = "CIUDAD"
    SUBCOMPACTO = "SUBCOMPACTO"
    COMPACTO = "COMPACTO"
    FAMILIAR = "FAMILIAR"
    EJECUTIVO = "EJECUTIVO"
    SUV = "SUV"
```

```
class TipoColor(Enum):
    BLANCO = "BLANCO"
    NEGRO = "NEGRO"
    ROJO = "ROJO"
    NARANJA = "NARANJA"
    AMARILLO = "AMARILLO"
    VERDE = "VERDE"
    AZUL = "AZUL"
    VIOLETA = "VIOLETA"
```

```
class Automovil:
    def __init__(self, marca, modelo, motor, tipo_combustible,
tipo_automovil, numero_puertas, cantidad_asientos, velocidad_maxima,
color, velocidad_actual):
        self.marca = marca
        self.modelo = modelo
        self.motor = motor
        self.tipo_combustible = tipo_combustible
        self.tipo_automovil = tipo_automovil
        self.numero_puertas = numero_puertas
        self.cantidad_asientos = cantidad_asientos
        self.velocidad_maxima = velocidad_maxima
        self.color = color
        self.velocidad_actual = velocidad_actual

    def getMarca(self):
        return self.marca

    def getModelo(self):
        return self.modelo

    def getMotor(self):
        return self.motor
```

```
def getTipoCombustible(self):
    return self.tipo_combustible

def getTipoAutomovil(self):
    return self.tipo_automovil

def getNumeroPuertas(self):
    return self.numero_puertas

def getCantidadAsientos(self):
    return self.cantidad_asientos

def getVelocidadMaxima(self):
    return self.velocidad_maxima

def getColor(self):
    return self.color

def getVelocidadActual(self):
    return self.velocidad_actual

def setMarca(self, marca):
    self.marca = marca

def setModelo(self, modelo):
    self.modelo = modelo

def setMotor(self, motor):
    self.motor = motor

def setTipoCombustible(self, tipo_combustible):
    self.tipo_combustible = tipo_combustible

def setTipoAutomovil(self, tipo_automovil):
    self.tipo_automovil = tipo_automovil

def setNumeroPuertas(self, numero_puertas):
    self.numero_puertas = numero_puertas

def setCantidadAsientos(self, cantidad_asientos):
    self.cantidad_asientos = cantidad_asientos
```

```

def setVelocidadMaxima(self, velocidad_maxima):
    self.velocidad_maxima = velocidad_maxima

def setColor(self, color):
    self.color = color

def setVelocidadActual(self, velocidad_actual):
    self.velocidad_actual = velocidad_actual

def acelerar(self, incremento_velocidad):
    if incremento_velocidad + self.velocidad_actual <=
self.velocidad_maxima:
        self.velocidad_actual += incremento_velocidad
    else:
        print(f'No es posible acelerar a mas de
{self.velocidad_maxima}')

def desacelerar(self, disminucion_velocidad):
    if self.velocidad_actual - disminucion_velocidad >= 0:
        self.velocidad_actual -= disminucion_velocidad
    else:
        print(f'No es posible desacelerar a menos de 0')

def frenar(self):
    self.setVelocidadActual(0)

def calcular_tiempo_llegada(self, distancia):
    return distancia / self.velocidad_actual if
self.velocidad_actual != 0 else None

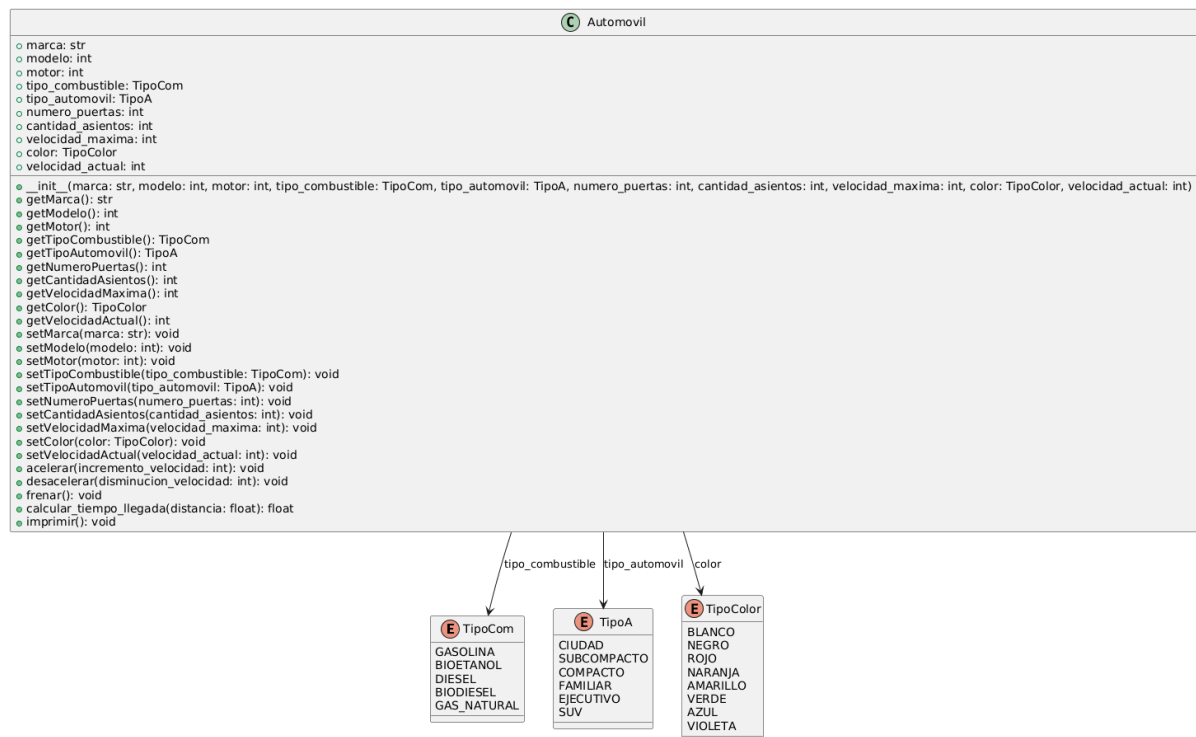
def imprimir(self):
    print(f'Marca = {self.marca}')
    print(f'Modelo = {self.modelo}')
    print(f'Motor = {self.motor}')
    print(f'Tipo de combustible = {self.tipo_combustible.name}')
    print(f'Tipo de automovil = {self.tipo_automovil.name}')
    print(f'Numero de puertas = {self.numero_puertas}')
    print(f'Cantidad de asientos = {self.cantidad_asientos}')
    print(f'Velocidad maxima = {self.velocidad_maxima}')
    print(f'Color = {self.color.name}')
    print(f'Velocidad actual = {self.velocidad_actual}')

```

```
def main():
    marcherati = Automovil('Nissan', 2023, 1600, TipoCom.GAS_NATURAL,
TipoA.EJECUTIVO, 5, 5, 320, TipoColor.BLANCO, 100)
    marcherati.imprimir()
    marcherati.acelerar(20)
    print(marcherati.getVelocidadActual())
    marcherati.desacelerar(50)
    print(marcherati.getVelocidadActual())
    marcherati.frenar()
    print(marcherati.getVelocidadActual())

if __name__ == "__main__":
    main()
```

### Diagrama de clase:



### Ejercicio 2.4

#### Codigo:

```
import math

class Circulo:
```

```

def __init__(self, radio):
    self.radio = radio

def calcular_area(self):
    return math.pi * math.pow(self.radio, 2)

def calcular_perimetro(self):
    return math.pi * 2 * self.radio

class Rectangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return self.base * self.altura

    def calcular_perimetro(self):
        return 2 * (self.base + self.altura)

class Cuadrado:
    def __init__(self, lado):
        self.lado = lado

    def calcular_area(self):
        return math.pow(self.lado, 2)

    def calcular_perimetro(self):
        return 4 * self.lado

class TrianguloRectangulo:
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return self.base * self.altura / 2

    def calcular_perimetro(self):

```

```

        return self.base + self.altura + math.sqrt(math.pow(self.base,
2) + math.pow(self.altura, 2))

    def calcular_hipotenusa(self):
        return math.sqrt(math.pow(self.base, 2) + math.pow(self.altura,
2))

    def determinar_tipo_triangulo(self):
        if self.base == self.altura and self.base ==
self.calcular_hipotenusa() and self.altura ==
self.calcular_hipotenusa():
            print('Es triangulo equilatero')
        elif self.base != self.altura and self.base !=
self.calcular_hipotenusa() and self.altura !=
self.calcular_hipotenusa():
            print('Es un triangulo escaleno')
        else:
            print('Es un triangulo isosceles')

def main():
    circulo = Circulo(2)
    print(f'El área del círculo es: {circulo.calcular_area()}')
    print(f'El perímetro del círculo es:
{circulo.calcular_perimetro()}')

    rectangulo = Rectangulo(1, 2)
    print(f'El área del rectángulo es: {rectangulo.calcular_area()}')
    print(f'El perímetro del rectángulo es:
{rectangulo.calcular_perimetro()}')

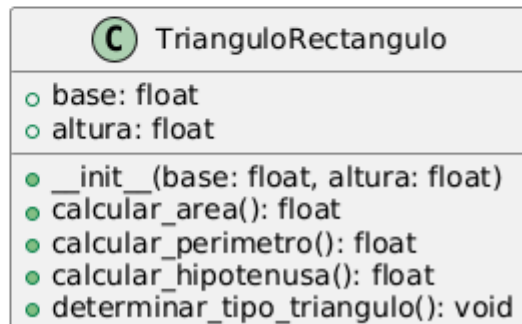
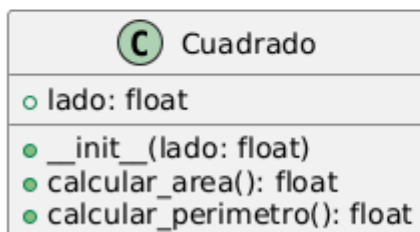
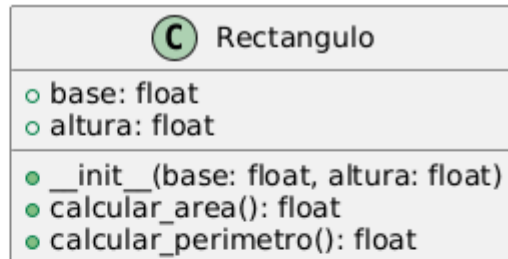
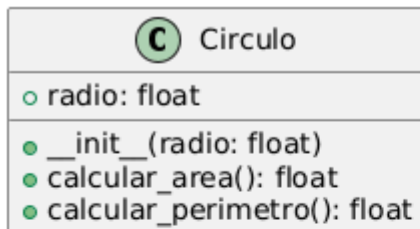
    cuadrado = Cuadrado(3)
    print(f'El área del cuadrado es: {cuadrado.calcular_area()}')
    print(f'El perímetro del cuadrado es:
{cuadrado.calcular_perimetro()}')

    triangulo = TrianguloRectangulo(3, 5)
    print(f'El área del triángulo es: {triangulo.calcular_area()}')
    print(f'El perímetro del triángulo es:
{triangulo.calcular_perimetro()}')
    print(f'La hipotenusa del triángulo es:
{triangulo.calcular_hipotenusa()}')
    triangulo.determinar_tipo_triangulo()

```

```
if __name__ == "__main__":
    main()
```

### Diagrama de clase:



### Ejercicio 2.5

#### Codigo:

```
from enum import Enum
```

```
class TipoCuenta(Enum):
    AHORROS = "AHORROS"
    CORRIENTE = "CORRIENTE"
```

```
class CuentaBancaria:
    def __init__(self, nombres_titular, apellidos_titular,
numero_cuenta, tipo_cuenta):
        self.nombres_titular = nombres_titular
        self.apellidos_titular = apellidos_titular
        self.numero_cuenta = numero_cuenta
```

```
self.tipo_cuenta = tipo_cuenta
self.saldo = 0.0

def imprimir(self):
    print(f'Nombres del titular = {self.nombres_titular}')
    print(f'Apellidos del titular = {self.apellidos_titular}')
    print(f'Número de cuenta = {self.numero_cuenta}')
    print(f'Tipo de cuenta = {self.tipo_cuenta.name}')
    print(f'Saldo = {self.saldo}')

def consultar_saldo(self):
    print(f'El saldo de la cuenta es ${self.saldo}')

def consignar(self, valor):
    if valor > 0:
        self.saldo += valor
        print(f'Se ha consignado ${valor} en la cuenta. El nuevo
saldo es ${self.saldo}')
        return True
    print('El valor a consignar debe ser mayor que 0')
    return False

def retirar(self, valor):
    if 0 < valor <= self.saldo:
        self.saldo -= valor
        print(f'Se ha retirado ${valor} en la cuenta. El nuevo
saldo es ${self.saldo}')
        return True
    print('No es posible retirar ese valor')
    return False

def comparar_cuentas(self, cuenta):
    if self.saldo > cuenta.saldo:
        print(f'La cuenta de {self.nombres_titular} tiene mayor
saldo que la de {cuenta.nombres_titular}')
    elif self.saldo < cuenta.saldo:
        print(f'La cuenta de {self.nombres_titular} tiene menor
saldo que la de {cuenta.nombres_titular}')
    else:
        print('Las dos cuentas tienen el mismo saldo')

def transferencia(self, cuenta, valor):
    if self.retirar(valor):
```

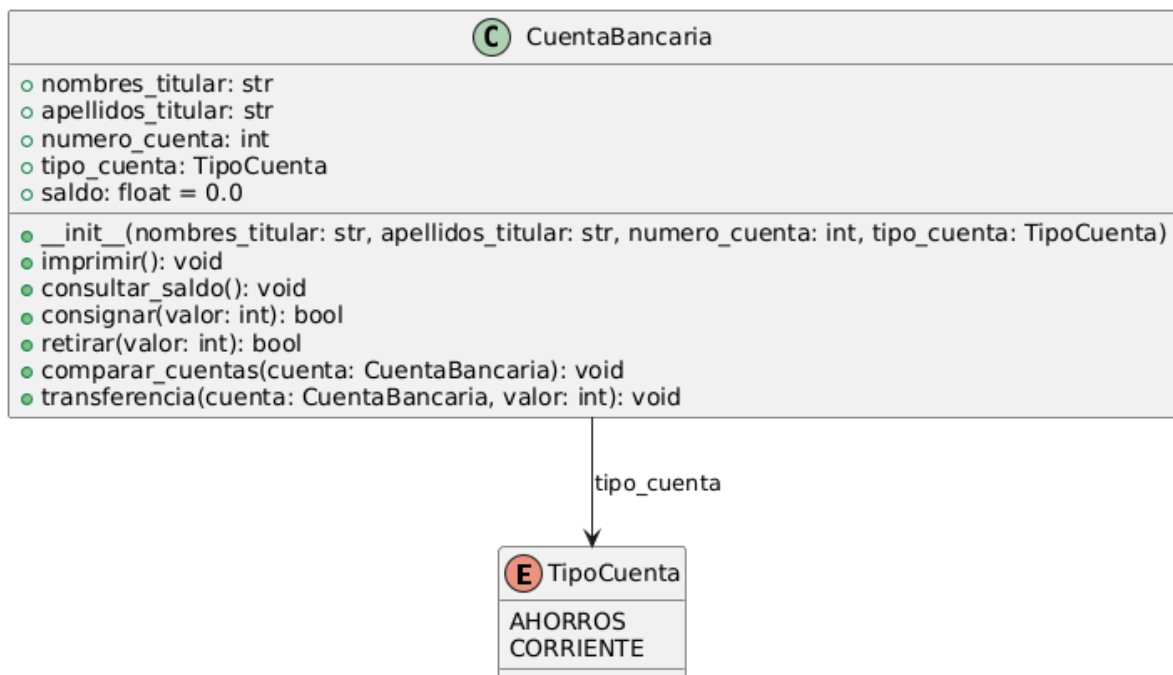
```
cuenta.consignar(valor)
```

```
def main():
    cuenta = CuentaBancaria('Pedro', 'Pérez', 123456789,
TipoCuenta.AHORROS)
    cuenta.imprimir()
    cuenta.consignar(200000)
    cuenta.consignar(300000)
    cuenta.retirar(400000)

    cuenta2 = CuentaBancaria('Ana', 'Gómez', 987654321,
TipoCuenta.CORRIENTE)
    cuenta.comparar_cuentas(cuenta2)
    cuenta.transferencia(cuenta2, 50000)
    cuenta2.consultar_saldo()

if __name__ == "__main__":
    main()
```

### Diagrama de clase:



### Repositorio GitHub

[Link](#)